Explainability in Reinforcement Learning

Maxime Alaarabiou

Sunday 30th August, 2026

Chapter 1

Reinforcement Learning

This chapter introduces the fundamental concepts of Reinforcement Learning (RL), providing the necessary foundation for the ideas developed throughout the rest of this manuscript. Its structure is as follows:

Section 1.1: An overview of the milestones that led to the emergence of RL as a unified field.

Section 1.2: A clarification of what is meant by an agent and how it relates to the RL algorithm.

Section 1.3: A formalization of the objective underlying RL algorithms.

Section 1.4: An explanation of how policies are progressively refined through interaction with the environment to approach the optimal policy.

Section 1.5: A summary of the key ideas and a discussion of the broader significance of RL.

1.1 Historical Developments

RL is today presented as a major field within artificial intelligence. However, it originates from several research traditions that progressively converged toward a common framework. In this section, we outline the main stages in the emergence of this field, highlighting the key ideas and milestones that shaped its evolution.

1.1.1 Behavioral foundations (1900–1950)

The origins of RL can be traced back to 20th-century animal psychology studies [Thorndike, 1911]. These studies aimed to understand how behaviors are formed and modified. The central idea emerging from this work is that learning is driven by the consequences of actions. Behaviors followed by positive outcomes are more likely to be repeated, whereas those associated with negative outcomes tend to gradually decrease in frequency.

This perspective was later developed within behaviorism. In this school of psychology, rewards and punishments are used to gradually shape behavior through repeated interaction with the environment [Skinner, 1938]. The term

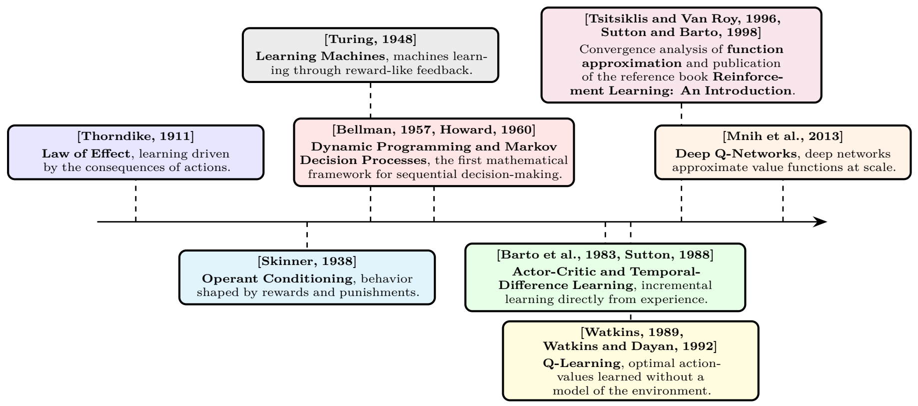

Figure 1.1: Evolution of reinforcement learning prior to its fusion with deep learning.

"reinforcement" in the context of learning comes from this school of thought, where it refers precisely to the process of strengthening a behavior through its consequences.

This idea led to the suggestion that learning principles observed in living organisms could be transferred to artificial systems. Machines were therefore envisioned as potentially learning through reward-like mechanisms, guided by evaluative feedback signals resembling a "pleasure–pain system" [Turing, 1948]. Although these studies did not involve formal mathematical models, they introduced the central idea of RL, trial-and-error learning guided by environmental feedback.

1.1.2 Sequential decision-making (1950–1970)

A second foundation of RL comes from optimal control theory. This field studies how to determine sequences of decisions that optimize a cumulative performance criterion over time. In this context, the decision-making entity is called an agent. The agent interacts with the environment by selecting actions. The function that maps an observed state to the chosen action is called a policy. The objective is to find an optimal policy, in the sense that it maximizes the expected cumulative reward over time.

This framework led to the formalization of Markov Decision Processes (MDP), which provide a mathematical model for sequential decision-making [Howard, 1960]. MDP describe a agent interacting with an environment through a set of states, actions, rewards, and transition rules. The earliest method proposed to solve MDP is dynamic programming [Bellman, 1957]. It proceeds by breaking down a complex decision problem into smaller, more manageable subproblems, solving each one, and combining the results to obtain an overall solution. However, dynamic programming requires a full and exact description of the environment, which is only feasible for small problems.

1.1.3 Emergence of reinforcement learning (1970–2000)

In order to address this issue, temporal-difference learning methods were introduced [Sutton, 1988]. This mechanism enables incremental learning directly from experience without requiring a complete model of the environment. It can be interpreted as a sample-based approximation of the Bellman equations.

Building on these ideas, the notion of value functions emerged as a way to estimate the long-term benefit of being in a given state or taking a given action. These functions assign a numerical score to states or state-action pairs, reflecting the expected cumulative reward an agent can obtain from that point onward. They thus provide a compact way to evaluate and compare different courses of action. This line of work culminated in Q-learning, which showed that optimal action-value functions can be learned directly from interaction with the environment, without requiring a model of its dynamics [Watkins, 1989, Watkins and Dayan, 1992].

The publication of the foundational reference work [Sutton and Barto, 1998] contributed to the formalization of RL as a coherent field. From that point onward, the field can be considered well established and widely recognized as a distinct area of research.

1.1.4 Consolidation of reinforcement learning (2000–2013)

A limitation of early algorithms such as Q-learning is their reliance on explicit state–action tables. Although these methods remove the need to know the full dynamics of the MDP, they still require storing a value for each possible state–action pair. As a result, the agent must effectively visit and maintain estimates for a potentially enormous number of states in the environment. This requirement becomes infeasible in large-scale MDP, where the state space is too large to be exhaustively explored or stored in memory.

To address this issue, research shifted toward more general representations of value functions and policies. Instead of maintaining explicit tables, function approximation methods were introduced. These allowed value functions and policies to be represented using parametric models such as linear approximators or perceptron-like architectures [Barto et al., 1983, Tsitsiklis and Van Roy, 1996]. This change enabled generalization across states, reducing the dependence on exhaustive exploration of the state space. The introduction of Deep Q-Networks (DQN), which demonstrated that deep Neural Network (NN) could be used to approximate value functions at scale [Mnih et al., 2013].

Figure 1.1 provides a timeline of the key milestones that shaped the development of reinforcement learning prior to its fusion with deep learning.

1.1.5 Deep reinforcement learning (2013–2026)

The introduction of Deep Learning (DL) into RL marked a major turning point in the field. A series of algorithms significantly improved scalability. Deep Deterministic Policy Gradient (DDPG) extended actor–critic methods to continuous action spaces [Lillicrap et al., 2015]. Proximal Policy Optimization (PPO) later became a standard algorithm in RL due to its simplicity, stability, and ability to perform in both continuous and discrete action spaces [Schulman et al., 2017].

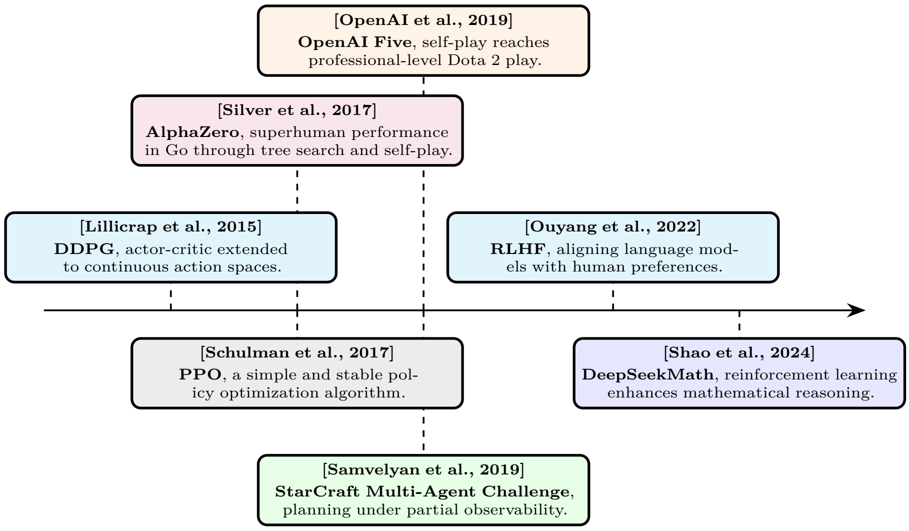

Figure 1.2: Evolution of deep reinforcement learning.

These advances established a practical foundation for large-scale policy optimization. Notably, AlphaZero demonstrated that the combination of deep NN, RL, and tree search could reach superhuman performance in the game of Go [Silver et al., 2017]. This success was later extended to even more complex real-time strategy environments such as StarCraft II, where agents must handle long-term planning, partial observability, and large action spaces [Samvelyan et al., 2019]. Similarly, in Dota 2, large-scale multi-agent RL systems such as OpenAI Five showed that self-play combined with deep policy optimization can achieve professional-level performance in highly dynamic and stochastic environments [OpenAI et al., 2019].

More recently, RL has expanded beyond its traditional application domains. With the emergence of increasingly agentic views of Large Language Model (LLM), it has become possible to train these models to perform specific tasks using RL-based techniques. In particular, RL from human feedback has become a method for aligning LLM with human preferences. It does so by formalizing the problem as an MDP and using RL to solve it [Ouyang et al., 2022]. Similarly, DeepSeekMath shows how RL can enhance the mathematical reasoning abilities of LLM [Shao et al., 2024]. It demonstrates that an artificial agent can learn to reason in natural language in order to accomplish a given task. The skills acquired during this learning process also transfer to other similar tasks, leading to improved overall performance. For instance, training a LLM with RL on mathematical problem-solving can also enhance its programming capabilities.

Figure 1.2 provides a timeline of the key milestones that shaped the emergence of deep reinforcement learning.

1.2 Agent Paradigm

The RL framework is built around the agent paradigm, in which the decision-making entity is referred to as the agent. A standard definition is:

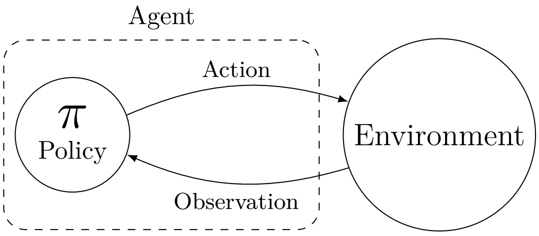

Figure 1.3: The agent paradigm.

> "An agent is anything that can be viewed as perceiving its environment through sensors and acting upon that environment through actuators."
> [Russell and Norvig, 2021]

In the RL setting, this definition takes the following form: The agent perceives its environment through observations. These observations are then processed by a decision-making mechanism referred to as the policy. The policy determines how the agent modifies its environment through the actions it selects. Figure 1.3 illustrates this interaction.

The objective of an RL agent is to maximize the cumulative reward it receives through its interactions with the environment. To achieve this objective, the agent's policy must progressively improve through experience. The procedure responsible for this learning process is the RL algorithm, which progressively modifies the policy based on the experience collected through interaction.

In this manuscript, the RL algorithm is considered as a process external to the agent. Other presentations of RL describe the agent as both the learner and the decision-making entity [Sutton and Barto, 1998]. We chose to distinguish the agent from the RL algorithm because this provides a more general definition of an agent.

An agent can therefore exist without learning, for example when its behavior is entirely specified by a set of programmed rules. Such an agent still perceives, processes information, and acts upon its environment, but its behavior is not the result of RL.

This distinction also makes the definition consistent with modern implementations of RL [Espeholt et al., 2018, Liang et al., 2018], in which the acting policy and the learning algorithm are separate software components. Several copies of the policy, each interacting with its own instance of the environment, collect experience in parallel, while a single learning algorithm uses this experience to update the shared policy parameters.

1.3 Objective

Now that the main developments shaping RL have been outlined, the formal definition of its objective can be introduced. In essence, RL aims to build a system that makes sequential decisions in order to maximize a cumulative reward signal over the long term. The purpose of this section is to translate this intuitive goal into a rigorous mathematical framework. To this end, the presentation proceeds step by step:

1. Modeling the RL problem as a Markov Decision Process.
2. Defining the notion of a policy, which formalizes the agent's behavior.
3. Describing the interaction between the agent and the environment through trajectories.
4. Introducing value functions to evaluate policies.
5. Defining the notion of an optimal policy.

1.3.1 Markov Decision Processes

To specify the task and the agent's capacity to perceive and act, the problem is modeled as a MDP.

Before providing a more detailed definition, it is useful to clarify the scope of the MDP problems considered in this manuscript. We therefore restrict the analysis to the following configuration:

- The single-agent learning setting is considered. If any other agents are present in the environment, they are treated as part of the environment dynamics.
- Finite-horizon MDP are considered. Each interaction sequence between the MDP and the policy has a defined beginning and a defined end. Between these two points, the number of interaction steps may vary depending on the situation, but it always remains finite.
- The partially observable framework is adopted, where the agent does not have direct access to the true state of the system, but instead relies on limited observations that provide only partial information about the environment.

The resulting framework can be formally described as a single-agent finite-horizon partially observable Markov decision process. For simplicity, we will refer to this setting as an "MDP" throughout this manuscript.

The following section aims to provide a mathematical description of the RL problem. The formulation adopted in this manuscript has been chosen to emphasize the main elements that will be used throughout the remainder of the manuscript. A MDP is defined by the tuple $(\mathcal{S}, \mathcal{O}, \mathcal{A}, \mathcal{R}, p)$:

- $\mathcal{S}$: The set of states, representing all possible configurations of the environment. A state belonging to this set is denoted by $s \in \mathcal{S}$. Among these, two special subsets are distinguished: initial states, from which an interaction can begin, and terminal states, in which the interaction ends. The set of initial states is denoted by $\mathcal{S}^{\text{init}}$, with an initial state written as $s^{\text{init}} \in \mathcal{S}^{\text{init}}$. The set of terminal states is denoted by $\mathcal{S}^{\text{ter}}$, with a terminal state written as $s^{\text{ter}} \in \mathcal{S}^{\text{ter}}$.

- $\mathcal{O}$: The set of observations that the agent can perceive from the environment. A observation belonging to this set is denoted by $o \in \mathcal{O}$. Unlike the true state $s \in \mathcal{S}$, which is hidden from the agent, the observation $o \in \mathcal{O}$ provides partial information about the current state.

- $\mathcal{A}$: The set of actions that the agent may execute. A action belonging to this set is denoted by $a \in \mathcal{A}$. Actions influence the dynamics of the MDP, thereby shaping its future course. They represent the agent's capacity to act upon its environment.

- $\mathcal{R}$: The set of rewards that the agent can receive from the environment. A reward belonging to this set is denoted by $r \in \mathcal{R}$. $\mathcal{R}$ is a subset of $\mathbb{R}$. Rewards can be either positive or negative, depending on whether the corresponding outcome is considered beneficial or detrimental for the agent. The agent's goal is to maximize the cumulative sum of these rewards over time.

- p: The transition function, which defines the dynamics of the environment. Given a state s and an action a, this function defines a probability distribution over the joint outcome consisting of the next state s', the observation o, and the reward r. This function is denoted by p, and is defined as: $p : \mathcal{S} \times \mathcal{A} \to \mathcal{P}(\mathcal{S} \times \mathcal{O} \times \mathcal{R})$. The notation $(s', o, r) \sim p(s, a)$ means that the tuple (s', o, r) is sampled from the distribution induced by $p(\cdot \mid s, a)$. The notation $p(s', o, r \mid s, a)$ denotes the probability of observing the tuple (s', o, r) under the distribution $p(\cdot \mid s, a)$.

The two quantities, observation o and state s, are not independent. They are jointly generated via $(s', o, r) \sim p(s, a)$, which implicitly defines the relationship between them. In our MDP formalization, this relationship is left implicit because it is never manipulated directly during learning.

However, to express mathematical equations, it is convenient to introduce the conditional distribution over states given an observation. We denote this distribution by $b : \mathcal{O} \to \mathcal{P}(\mathcal{S})$. The notation $b(s \mid o)$ denotes the probability of state s given the observation o, while $s \sim b(\cdot \mid o)$ indicates that the state s is sampled from this distribution.

1.3.2 Policy

The agent's decision-making capability is modeled by a policy function. The policy defines the stochastic mapping between observations $o \in \mathcal{O}$ and actions

$a \in \mathcal{A}$. A policy function is denoted by π, and is defined as: $\pi : \mathcal{O} \to \mathcal{P}(\mathcal{A})$. The notation $a \sim \pi(\cdot \mid o)$ means that the action a is sampled from the distribution induced by $\pi(\cdot \mid o)$. The notation $\pi(a \mid o)$ denotes the probability of selecting action a under the distribution $\pi(\cdot \mid o)$.

It is important to note that this policy does not include any explicit mechanism for estimating the underlying state from the observations. Instead, the policy must learn from experience how to infer the information about the underlying state that is not directly available in the observations. This reflects the design of many modern RL implementations, where policies are often parameterized by architectures capable of summarizing past observations, such as recurrent or attention-based networks, without explicitly modeling the uncertainty over the underlying state.

1.3.3 Trajectory

Now that both the MDP $(\mathcal{S}, \mathcal{O}, \mathcal{A}, \mathcal{R}, p)$ and the policy π have been defined, the interaction between them can be characterized. A sequence of consecutive interactions between an environment and a policy is called a trajectory. A trajectory is generated through the repeated interaction between the policy and the environment, following the sequence below:

1. The environment, being in state s, provides an observation o to the policy π. Based on this observation, the policy outputs a probability distribution over actions $\pi(\cdot \mid o)$, from which an action is sampled: $a \sim \pi(\cdot \mid o)$.

2. The sampled action is executed in the environment. The environment then generates a transition: $(s', o', r) \sim p(\cdot \mid s, a)$. It moves to a new state s', and returns a new observation o' and a reward r.

A trajectory t of length T can be written as:

$$t = (s_1, o_1, a_1, r_1, s_2, o_2, a_2, r_2, \ldots, s_T, o_T, a_T, r_T).$$

It is important to clearly distinguish between a trajectory distribution and its realizations. The distribution over trajectories is denoted by τ, and a particular trajectory is denoted by t. The distribution τ is determined by the initial state s, the initial observation o, the environment dynamics p, and the policy π, and is denoted by $\tau(s, o, p, \pi)$. The notation $t \sim \tau(s, o, p, \pi)$ means that the trajectory t is sampled from the distribution induced by s, o, p, and π. The notation $\tau(t \mid s, o, p, \pi)$ denotes the probability of observing the trajectorie t under the distribution $\tau(\cdot \mid s, o, p, \pi)$

An important property of the trajectory distribution is its recursive structure. Since both the policy π and the transition function p are known, the distribution over trajectories starting from s_1 with observation o_1 can be expressed in terms of the distribution over trajectories starting from the next state s_2 with observation o_2. Specifically, the probability of a trajectory t factorizes as:

$$\tau(t \mid s_1, o_1, p, \pi) = \pi(a_1 \mid o_1)\, p(s_2, o_2, r_1 \mid s_1, a_1)\, \tau(t_{2:} \mid s_2, o_2, p, \pi),$$

where $t_{2:} = (s_2, o_2, a_2, r_2, \ldots, s_T, o_T, a_T, r_T)$ denotes the trajectory starting from the second step.

There exist two special types of trajectories. First, a trajectory that terminates in a terminal state is called a terminal trajectory, and is denoted by $t_t \sim \tau_t(s, o, p, \pi)$. Second, a terminal trajectory that starts from an initial state and an initial observation is called an episode, and is denoted by $t_e \sim \tau_t(s^{\text{init}}, o, p, \pi)$.

1.3.4 Value Functions

To evaluate the ability of a policy π to collect rewards, the notion of return is introduced. The return of a trajectory $g(t, \gamma)$, is defined as the discounted sum of rewards collected along the trajectory:

$$g(t, \gamma) = r_1 + \gamma r_2 + \gamma^2 r_3 + \cdots + \gamma^{T-1} r_T = \sum_{i=1}^{T} \gamma^{i-1} r_i,$$

where $\gamma \in [0, 1]$ is the discount factor. This parameter controls the time horizon of the agent's objective. When γ is close to 0, the agent focuses almost exclusively on immediate rewards, leading to short-sighted behavior. As γ increases toward 1, future rewards gain more weight, and the agent adopts a more far-sighted strategy. In theory, since the goal is to maximize long-term cumulative reward, the discount factor would ideally be set to $\gamma = 1$. In practice, however, γ is often chosen slightly below 1 (e.g., 0.99) to stabilize learning, and to reflect the fact that distant future rewards are inherently more uncertain.

The return also admits a recursive form, which is central to the dynamic programming and RL algorithms presented later:

$$g(t, \gamma) = r_1 + \gamma\, g(t_{2:}, \gamma),$$

From the notion of return, functions that estimate the expected return of a policy from a given starting point can be defined. These functions are called value functions. The first one considered is the state-value function V. It tells us how desirable a particular state is under a given policy π, given an observation o, the environment dynamics p, and the discount factor γ. Strictly speaking, this function should be written as $V(s, o, p, \gamma, \pi)$ to reflect all its dependencies. However, since both the environment dynamics p and the discount factor γ are fixed throughout learning, they are omitted from the notation. To emphasize that the value is tied to a specific policy, π is placed as a subscript. This yields the compact notation $V_\pi : \mathcal{S} \times \mathcal{O} \to \mathbb{R}$.

If $V_\pi(s, o) > V_\pi(s', o')$, then, under policy π, the agent prefers being in state s with observation o over state s' with observation o'. Formally, the state-value function is defined as:

$$V_\pi(s, o) = \mathbb{E}\Big[g(t, \gamma)\Big], \quad \text{with } t \sim \tau_t(s, o, p, \pi).$$

It is worth noting that the state-value function V_π is recursive, a property that follows directly from the recursive form of the return:

$$\begin{aligned} V_\pi(s, o) &= \mathbb{E}\Big[g(t, \gamma)\Big], \quad \text{with } t \sim \tau_t(s, o, p, \pi) \\ &= \mathbb{E}\Big[r_1 + \gamma\, g(t_{2:}, \gamma)\Big] \\ &= \mathbb{E}\Big[r_1 + \gamma\, V_\pi(s_2)\Big] \end{aligned}$$

If the expectation is expressed explicitly in terms of the distributions p and π, the Bellman equation for V_π is obtained:

$$\begin{aligned} V_\pi(s,o) &= \mathbb{E}\Big[r_1 + \gamma\, V_\pi(s_2)\Big] \\ &= \sum_{a\in\mathcal{A}} \pi(a \mid o) \sum_{\substack{s'\in\mathcal{S}\\ o'\in\mathcal{O}\\ r\in\mathcal{R}}} p(s',o',r \mid s,a)\Big[r + \gamma\, V_\pi(s',o')\Big]. \end{aligned}$$

Now that the value of a state has been expressed, the next step is to consider the value of taking a specific action in a given state. Indeed, it is useful to estimate the expected future gain of an action a in state s, in order to compare different actions and determine which one is the most promising under a given policy π. This function is called the state-action value function $Q_\pi : \mathcal{S}\times\mathcal{A}\to\mathbb{R}$. If $Q_\pi(s,a) > Q_\pi(s,a')$, then, under policy π, choosing action a in state s yields a higher expected return than choosing action a'. It is defined as:

$$Q_\pi(s,a) = \mathbb{E}\Big[r + \gamma\, g(t,\gamma)\Big], \quad \text{with } t\sim\tau_t(s',o,p,\pi), \quad (s',o,r)\sim p(\cdot \mid s,a).$$

The function Q_π can be related to V_π by recognizing that, after taking action a in state s, the remaining return from the next state s' is precisely $V_\pi(s')$. This yields:

$$\begin{aligned} Q_\pi(s,a) &= \mathbb{E}\Big[r + \gamma\, g(t,\gamma)\Big], \quad \text{with } t\sim\tau_t(s',o,p,\pi), \quad (s',o,r)\sim p(\cdot \mid s,a) \\ &= \mathbb{E}\Big[r + \gamma\, V_\pi(s',o)\Big] \\ &= \sum_{\substack{s'\in\mathcal{S}\\ o'\in\mathcal{O}\\ r\in\mathcal{R}}} p(s',o',r \mid s,a)\Big[r + \gamma\, V_\pi(s',o)\Big] \end{aligned}$$

1.3.5 Optimal Policy

The goal of RL is to find a policy that maximizes the expected cumulative reward from any observation until the end of the trajectory. This policy is called the optimal policy and is denoted by π^*. In a partially observable setting, the optimal action is the one that maximizes the expected state-action value over the distribution of possible states given the observation:

$$\pi^*(a \mid o) = \begin{cases} 1 & \text{if } a = \underset{a'\in\mathcal{A}}{\arg\max}\ \sum_{s\in\mathcal{S}} b(s \mid o)\, Q_{\pi^*}(s,a'), \\ 0 & \text{otherwise} \end{cases}$$

As shown, both V_π and Q_π can be expressed explicitly in terms of the environment dynamics p and the policy π. If p, b and π are fully known, one can theoretically compute the optimal policy by unfolding the graph of all possible trajectories starting from a given state s and observation o. This is possible precisely because V_π and Q_π are recursive functions. Solving an MDP in this way is known as dynamic programming.

In practice, most environments of have too many states to allow an explicit representation of the dynamics p. Storing and computing over the full transition function becomes intractable. To overcome this limitation, instead of directly constructing the optimal policy, an iterative approach is adopted: starting from an initial policy, it is progressively improved so that it converges toward π^*. This iterative improvement through interaction with the environment is precisely what constitutes learning in RL, and it is the subject of the next section.

1.4 Learning

Now that the optimal policy π^* has been defined, the question is how to obtain it. The recursive Bellman equations for V_π and Q_π suggest dynamic programming approaches, but these scale poorly to large state spaces. RL offers an alternative by solving the MDP iteratively and generating a sequence of policies that progressively approach π^* through direct interaction with the environment. This section introduces two major families of RL algorithms, critic-only methods and actor-critic methods.

At a high level, every algorithm presented below repeats the same loop: the current policy interacts with the environment, the resulting trajectories are sent to the RL algorithm, and the algorithm uses them to produce a new policy. The two families differ only in how this last step is carried out.

1.4.1 Critic-Only Methods

The central idea behind all critic-only approaches is to iteratively construct an approximation of the state-action value function Q_π, denoted by $\hat{Q}_\pi$. Although $\hat{Q}_\pi$ is only an approximation of the true Q_π, the policy always acts greedily with respect to this approximation.

Since $\hat{Q}_\pi$ serves as the basis for the agent's decisions, it cannot rely on more information than the agent itself has access to. It must therefore be defined solely over observations, as a function $\hat{Q}_\pi : \mathcal{O} \times \mathcal{A} \to \mathbb{R}$. Its objective is to approximate the true Q_π, which depends on the underlying states s, using only the partial information available through o. In practice, this does not necessarily pose a problem, as observations often contain sufficient information to make accurate predictions about the underlying state. Moreover, the strong representational capacity of NN allows the critic to capture these relationships without explicitly representing the mapping between observations and states.

This leads to the following mathematical formulation:

$$\pi(a \mid o) = \begin{cases} 1 & \text{if } a = \arg\max_{a' \in \mathcal{A}} \hat{Q}_\pi(o, a'), \\ 0 & \text{otherwise.} \end{cases} \tag{1.1}$$

It is precisely this way of modeling the policy that gives this family of methods its name. The policy is not represented explicitly as a standalone function. Rather, it simply emerges from searching over the set of actions to maximize $\hat{Q}_\pi$.

To give a clear example of critic-only methods, they are presented in their simplest form: model-free, without temporal-difference learning, without replay buffer, without bootstrapping, and using only ε-greedy exploration. Naturally, many variants and extensions of this basic scheme exist, but the core idea remains the same.

When implementing a RL algorithm, one of the first questions is how to represent the value function. In critic-only methods, this means choosing a model for $\hat{Q}_\pi$. Any machine learning model capable of regression task can be used for this purpose. The set of parameters of the model is denoted by θ. In deep RL, a NN is used to model this function, in which case θ represents the weights and biases of the network.

In our simple setting, exploration is handled via ε-greedy selection. With probability $1-\varepsilon$, the policy selects the greedy action that maximizes $\hat{Q}_\pi$, and with probability ε it selects an action uniformly at random. Formally, this ε-greedy policy is obtained by modifying the greedy policy defined above as follows:

$$\pi(a \mid o) = \begin{cases} 1 - \varepsilon + \dfrac{\varepsilon}{|\mathcal{A}|} & \text{if } a = \underset{a' \in \mathcal{A}}{\arg\max}\, \hat{Q}_\pi(o, a'), \\ \dfrac{\varepsilon}{|\mathcal{A}|} & \text{otherwise.} \end{cases} \tag{1.2}$$

The trajectory distribution induced by the resulting policy now depends on both π and ε, and is denoted by $t \sim \tau(s, o, p, \pi, \varepsilon)$. The parameter ε controls the degree of exploration. If the policy always chooses the action it currently believes to be optimal, it may miss better alternatives that $\hat{Q}_\pi$ has not yet learned to value correctly. Exploring suboptimal actions from time to time is therefore essential to verify that $\hat{Q}_\pi$ accurately reflects the true values, while still focusing most of the time on the most promising actions. In more advanced implementations, ε can be adapted over the course of learning to control exploration dynamically, for instance by decreasing it gradually over time.

The learning process consists of two steps that repeat until convergence:

1. **Data collection.** The current policy π interacts with the MDP. For a given starting state s_1^{init} and observation o_1, the interaction produces a episode t_e sampled from $\tau_t(s_1^{\text{init}}, o_1, p, \pi, \varepsilon)$. For each step i along the episode, the return from that step onward is $g(t_{i:}) = \sum_{k=i}^{T} \gamma^{k-i} r_k$, which serves as the target value for the state-action pair (s_i, a_i). This procedure is detailed in Algorithm 1.

2. **Model update.** The tuples $(s_i, a_i, g(t_{i:}))$ obtained in the previous step serve as supervised training data: the state-action pair (s_i, a_i) is the input, and the return $g(t_{i:})$ is the associated target. These examples are used to train $\hat{Q}_\theta$. The improved approximation is denoted by $\hat{Q}'_\theta$, which now predicts returns more accurately under π. Since $\hat{Q}'_\theta$ has changed, the policy derived from it also changes. This new policy is denoted by π'.

Because the policy is now π', the approximation $\hat{Q}'_\theta$ must be trained to predict returns under π'. This requires returning to step 1 to collect fresh interaction data generated by π'. The two steps thus alternate until the policy and the value approximation stabilize. Algorithm 2 summarizes this whole critic-only learning loop.

Even this simple algorithm already illustrates a difficulty inherent to all forms of RL: the instability of the learning process. Each update from $\hat{Q}_\theta$ to $\hat{Q}'_\theta$ causes the policy to shift from π to π'. For the learning to remain stable, the change from $\hat{Q}_\theta$ to $\hat{Q}'_\theta$ should be as small as possible. This way, $\hat{Q}'_\theta$, trained to predict returns under π, remains reasonably accurate under π'. However, the smaller the change, the slower the learning progresses. A trade-off must therefore be

Algorithm 1: Data Collection

Input:
MDP $(\mathcal{S}, \mathcal{O}, \mathcal{A}, \mathcal{R}, p)$
Policy π
Discount factor $\gamma \in [0, 1]$
Output:
Dataset $\mathcal{D}$

Sample an initial state.
$i \leftarrow 1$;
Sample $s_i^{\text{init}} \in \mathcal{S}^{\text{init}}$;
Observe o_i;

Episode generation by interaction with the MDP.
repeat
  Action selection.
  Sample $a_i \sim \pi(\cdot \mid o_i)$;
  Environment transition.
  Sample $(s_{i+1}, o_{i+1}, r_i) \sim p(\cdot \mid s_i, a_i)$;
  $i \leftarrow i + 1$;
until $s_i \in \mathcal{S}^{ter}$;

Return computation for each step of the trajectory.
$T \leftarrow i - 1$;
for $i = 1, \ldots, T$ **do**
  $g_i \leftarrow \sum_{k=i}^{T} \gamma^{k-i} r_k$;

Dataset construction
$\mathcal{D} \leftarrow \left\{ \left(s_i, o_i, a_i, g_i \right) \right\}_{i=1}^{T}$;
Return $\mathcal{D}$

struck between stability and learning speed. Many improvements introduced later in the literature aim at stabilizing this learning loop [Hessel et al., 2017].

Convergence guarantees for such algorithms have been proved under a different set of assumptions than the setting presented here. For example, in the classical convergence results, the MDP is fully observable, the value function is stored in a table, and the learning rate is decreased at a suitable rate over time [Watkins and Dayan, 1992]. In modern implementations, the value function is approximated by a NN. Theoretical convergence proofs no longer hold in this setting. Nevertheless, deep methods have demonstrated remarkable empirical success, making them the tool of choice for RL tasks.

Critic-only approaches suffer from several limitations. The main limitation is scalability to large action spaces. Since the policy must evaluate all possible actions at each decision step to select the maximum, the computational cost grows with the size of the action set. To address this issue, another family of RL algorithms exists: actor-critic methods.

Algorithm 2: Deep Critic-Only Reinforcement Learning

Input:
MDP $(\mathcal{S}, \mathcal{O}, \mathcal{A}, \mathcal{R}, p)$
Architecture $\hat{Q}$
Exploration rate $\varepsilon \in [0, 1]$
Discount factor $\gamma \in [0, 1]$
Learning rate $\eta > 0$
Initial parameters θ (optional)
Output:
Trained policy π_θ

---

if θ *is not given* **then**
  Initialize θ randomly;

Exploration policy as defined in Equation (1.2)
$\pi_\theta \leftarrow \varepsilon$-greedy policy with respect to $\hat{Q}_\theta$

MSE loss function as defined in Equation (**??**)
$\mathcal{L} \leftarrow$ Mean Squared Error (MSE);

while *stopping criterion not satisfied* **do**
  Data collection see Algorithm 1
  $\mathcal{D} \leftarrow$ `DataCollection(`*MDP*, π_θ, γ`)`

  observation-action pairs and their returns, from $\mathcal{D}$
  $\mathcal{D}' \leftarrow \left\{ \big((o, a), g \big) \mid (s, o, a, g) \in \mathcal{D} \right\}$

  Model update whith SGD see Algorithm **??**
  $\theta \leftarrow$ `SGD(`$\mathcal{D}', \hat{Q}, \mathcal{L}, \eta, \theta$`)`

Final policy without exploration as defined in Equation (1.1)
$\pi_\theta \leftarrow$ policy with respect to $\hat{Q}_\theta$;

Return π_θ

1.4.2 Actor-Critic Methods

Two principal modifications distinguish actor-critic methods from critic-only approaches. First, the objective is to approximate the state-value function V, instead of the state-action value function Q. Second, the policy π is given its own dedicated set of parameters. Two separate components are thus obtained: a value function approximation $\hat{V}_{\theta_1}$ and a parameterized policy π_{θ_2}.

Analogously to critic-only methods, $\hat{V}_{\theta_1}$ is trained using supervised learning on pairs obtained from interaction with the MDP. For each step i along a trajectory, the input is the state-observation pair (s_i, o_i) and the target is the observed return $g(t_{i:})$. The value function is thus trained to correct its predictions based on data collected by the current policy interacting with the environment.

The key difference lies in how the policy π itself is optimized. Since the policy now has its own set of parameters θ_2, it can be updated directly. The update uses the notion of advantage $A : \mathcal{S} \times \mathcal{A} \to \mathcal{R}$, defined as:

$$A(s, a) = g(t) - \hat{V}_{\theta_1}(s, o),$$

where t is the trajectory that starts by taking action a in state s and then follows π for all subsequent steps.

When $A(s, a) > 0$, the action a performed better than expected, and the policy parameters θ_2 are adjusted to increase the probability $\pi_{\theta_2}(a \mid o)$. When $A(s, a) < 0$, the action a performed worse than expected, and the probability is decreased.

Both $\hat{V}_{\theta_1}$ and π_{θ_2} are improved simultaneously within the same loop: the critic provides a learned baseline for the actor, and the actor generates the data used to train the critic. This interplay reduces the instability inherent to RL. Algorithm 3 summarizes this whole actor-critic learning loop.

PPO (Proximal Policy Optimization), which is the current standard in RL, belongs to this family of actor-critic methods. It will be used throughout this manuscript for all RL experiments.

1.5 Conclusion

This chapter has provided an introduction to the fundamental principles of RL. As in Chapter **??**, only the essential concepts required to understand the deep RL framework have been presented. The broader field of RL encompasses many additional methods and extensions.

Despite this limited scope, the defining characteristics of RL can already be observed in its simple learning procedure. A policy is iteratively improved using estimates obtained through the interaction between an agent and its environment. Learning therefore emerges from the repeated alternation of two elementary operations, which are performed until the policy and its associated value estimates converge.

This simple iterative structure brings RL conceptually close to DL. In both paradigms, simple operations repeated at scale can give rise to highly complex capabilities with limited human intervention. Their combination is particularly powerful. RL provides a general framework for learning through interaction, where a wide range of decision-making problems can be formulated as MDPs. DL, in turn, provides flexible function approximators capable of representing the value functions and policies required by this framework. Together, these two paradigms form the foundation of deep RL, whose empirical performance has led to remarkable results.

Originally inspired by biological learning processes, RL is now closer than ever to this initial intuition. Agents learn through interaction, adapt via trial and error, and progressively improve their behavior based on experience. This behavior is governed by deep neural architectures, which are the closest computational analog to organic brains. More than ever, RL stands as one of the core paradigms for achieving artificial general intelligence [Silver et al., 2021].

As the capabilities of RL continue to improve, a new challenge becomes increasingly important. The ability to learn complex behaviors with limited human intervention can make the resulting decisions difficult to understand. To address this challenge, a parallel research direction has consequently emerged. This is the subject of the next chapter, which focuses on explainability in the context of RL.

Algorithm 3: Deep Actor-Critic Reinforcement Learning

Input:
MDP $(\mathcal{S}, \mathcal{O}, \mathcal{A}, \mathcal{R}, p)$
Architecture $\hat{V}$ (critic)
Architecture π (actor)
Discount factor $\gamma \in [0, 1]$
Learning rate $\eta > 0$
Initial parameters θ_1 of the critic (optional)
Initial parameters θ_2 of the actor (optional)
Output:
Trained policy π_{θ_2}

```
if θ1 is not given then
    Initialize θ1 randomly;
if θ2 is not given then
    Initialize θ2 randomly;
// MSE loss function as defined in Equation (??)
L ← MSE;
while stopping criterion not satisfied do
    // Data collection see Algorithm 1
    D ← DataCollection(MDP, π_θ2, γ)
    // Advantage estimation: how much better/worse each action performed compared to the critic's expectation
    for (s_i, o_i, a_i, g_i) ∈ D do
        A(s_i, a_i) ← g_i − V̂_θ1(s_i, o_i);
    // state and their returns, from D
    D' ← {(s, g) | (s, o, a, g) ∈ D}
    // Critic update with SGD see Algorithm ??
    θ1 ← SGD(D', V̂, L, η, θ1)
    // Actor update: increase the probability of actions with positive advantage, decrease it for those with negative advantage
    θ2 ← θ2 + η Σ_{(s_i,o_i,a_i,g_i)∈D} A(s_i, a_i) ∇_θ2 log π_θ2(a_i | o_i);
Return π_θ2
```

Line 12 in math form:

$$\theta_2 \leftarrow \theta_2 + \eta \sum_{(s_i,o_i,a_i,g_i)\in\mathcal{D}} A(s_i,a_i)\,\nabla_{\theta_2}\log\pi_{\theta_2}(a_i \mid o_i);$$

Acronyms

DDPG Deep Deterministic Policy Gradient 3
DL Deep Learning 3, 15
DQN Deep Q-Networks 3

LLM Large Language Model 4

MDP Markov Decision Processes 2–4, 6–8, 10–16
MSE Mean Squared Error 14, 16

NN Neural Network 3, 4, 11–13

PPO Proximal Policy Optimization 3

RL Reinforcement Learning 1–13, 15

Bibliography

[Barto et al., 1983] Barto, A. G., Sutton, R. S., and Anderson, C. W. (1983). Neuronlike adaptive elements that can solve difficult learning control problems. IEEE Transactions on Systems, Man, and Cybernetics, 13(5):835–846.

[Bellman, 1957] Bellman, R. E. (1957). Dynamic programming.

[Espeholt et al., 2018] Espeholt, L., Soyer, H., Munos, R., Simonyan, K., Mnih, V., Ward, T., Doron, Y., Firoiu, V., Harley, T., Dunning, I., Legg, S., and Kavukcuoglu, K. (2018). IMPALA: Scalable distributed deep-RL with importance weighted actor-learner architectures. In Proceedings of the 35th International Conference on Machine Learning (ICML).

[Hessel et al., 2017] Hessel, M., Modayil, J., van Hasselt, H., Schaul, T., Ostrovski, G., Dabney, W., Horgan, D., Piot, B., Azar, M., and Silver, D. (2017). Rainbow: Combining improvements in deep reinforcement learning.

[Howard, 1960] Howard, R. A. (1960). Dynamic programming and markov processes. MIT Press.

[Liang et al., 2018] Liang, E., Liaw, R., Nishihara, R., Moritz, P., Fox, R., Goldberg, K., Gonzalez, J. E., Jordan, M. I., and Stoica, I. (2018). RLlib: Abstractions for distributed reinforcement learning. In Proceedings of the 35th International Conference on Machine Learning (ICML).

[Lillicrap et al., 2015] Lillicrap, T. P., Hunt, J. J., Pritzel, A., Heess, N., Erez, T., Tassa, Y., Silver, D., and Wierstra, D. (2015). Continuous control with deep reinforcement learning.

[Mnih et al., 2013] Mnih, V., Kavukcuoglu, K., Silver, D., Graves, A., Antonoglou, I., Wierstra, D., and Riedmiller, M. (2013). Playing atari with deep reinforcement learning.

[OpenAI et al., 2019] OpenAI, :, Berner, C., Brockman, G., Chan, B., Cheung, V., Dębiak, P., Dennison, C., Farhi, D., Fischer, Q., Hashme, S., Hesse, C., Józefowicz, R., Gray, S., Olsson, C., Pachocki, J., Petrov, M., d. O. Pinto, H. P., Raiman, J., Salimans, T., Schlatter, J., Schneider, J., Sidor, S., Sutskever, I., Tang, J., Wolski, F., and Zhang, S. (2019). Dota 2 with large scale deep reinforcement learning.

[Ouyang et al., 2022] Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C. L., Mishkin, P., Zhang, C., Agarwal, S., Slama, K., Ray, A., Schulman, J., Hilton, J., Kelton, F., Miller, L., Simens, M., Askell, A., Welinder, P., Christiano, P., Leike, J., and Lowe, R. (2022). Training language models to follow instructions with human feedback.

[Russell and Norvig, 2021] Russell, S. J. and Norvig, P. (2021). Artificial Intelligence: A Modern Approach. Pearson, 4th edition.

[Samvelyan et al., 2019] Samvelyan, M., Rashid, T., de Witt, C. S., Farquhar, G., Nardelli, N., Rudner, T. G. J., Hung, C.-M., Torr, P. H. S., Foerster, J., and Whiteson, S. (2019). The starcraft multi-agent challenge.

[Schulman et al., 2017] Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and Klimov, O. (2017). Proximal policy optimization algorithms.

[Shao et al., 2024] Shao, Z., Wang, P., Zhu, Q., Xu, R., Song, J., Bi, X., Zhang, H., Zhang, M., Li, Y. K., Wu, Y., and Guo, D. (2024). Deepseekmath: Pushing the limits of mathematical reasoning in open language models.

[Silver et al., 2017] Silver, D., Hubert, T., Schrittwieser, J., Antonoglou, I., Lai, M., Guez, A., Lanctot, M., Sifre, L., Kumaran, D., Graepel, T., Lillicrap, T., Simonyan, K., and Hassabis, D. (2017). Mastering chess and shogi by self-play with a general reinforcement learning algorithm.

[Silver et al., 2021] Silver, D., Singh, S., Precup, D., and Sutton, R. S. (2021). Reward is enough. Artificial intelligence, 299:103535.

[Skinner, 1938] Skinner, B. F. (1938). The behavior of organisms: An experimental analysis. Appleton-Century.

[Sutton, 1988] Sutton, R. S. (1988). Learning to predict by the methods of temporal differences. Machine Learning, 3:9–44.

[Sutton and Barto, 1998] Sutton, R. S. and Barto, A. G. (1998). Reinforcement Learning: An Introduction. MIT Press, 1st edition.

[Thorndike, 1911] Thorndike, E. L. (1911). Animal intelligence: Experimental studies. The Psychological Review Monograph Supplement, 8(4).

[Tsitsiklis and Van Roy, 1996] Tsitsiklis, J. and Van Roy, B. (1996). Analysis of temporal-diffference learning with function approximation. Advances in neural information processing systems, 9.

[Turing, 1948] Turing, A. (1948). Intelligent machinery. National Physical Laboratory Report.

[Watkins, 1989] Watkins, C. J. C. H. (1989). Learning from delayed rewards. PhD Thesis, University of Cambridge.

[Watkins and Dayan, 1992] Watkins, C. J. C. H. and Dayan, P. (1992). Q-learning. Machine Learning, 8:279–292.